

BubbleSort

Pseudocode (IHK-Stil):

```
funktion bubbleSort(A: Array<Integer>)  
  n = länge(A)  
  für i = 0 bis n-2  
    für j = 0 bis n-i-2  
      wenn A[j] > A[j+1] dann  
        tausche A[j], A[j+1]  
end funktion
```

Java-Beispiel:

```
public class BubbleSortDemo {  
  static void bubbleSort(int[] A) {  
    int n = A.length;  
    for (int i = 0; i < n-1; i++) {  
      for (int j = 0; j < n-i-1; j++) {  
        if (A[j] > A[j+1]) {  
          int temp = A[j];  
          A[j] = A[j+1];  
          A[j+1] = temp;  
        }  
      }  
    }  
  }  
}
```

BubbleSort – Erklärung & Beispiele

Beim BubbleSort werden benachbarte Elemente wiederholt verglichen und ggf. vertauscht. Das größte Element „wandert“ pro Durchlauf nach rechts.

****Komplexität:**** $O(n^2)$.

****Prüfungstipp:**** Wird oft als Basisalgorithmus abgefragt.

****Typische Fehlerquellen:****

- falsche Schleifen-Grenzen
- Vertauschen vergessen

Eingabe	Durchlauf 1	Durchlauf 2	Ergebnis
[5,3,1]	[3,1,5]	[1,3,5]	[1,3,5]

InsertionSort

Pseudocode (IHK-Stil):

```
funktion insertionSort(A: Array<Integer>)  
    für i = 1 bis länge(A)-1  
        key = A[i]  
        j = i-1  
        solange j ≥ 0 und A[j] > key  
            A[j+1] = A[j]  
            j = j - 1  
        A[j+1] = key  
end funktion
```

Java-Beispiel:

```
public class InsertionSortDemo {  
    static void insertionSort(int[] A) {  
        for (int i = 1; i < A.length; i++) {  
            int key = A[i];  
            int j = i-1;  
            while (j >= 0 && A[j] > key) {  
                A[j+1] = A[j];  
                j--;  
            }  
            A[j+1] = key;  
        }  
    }  
}
```

InsertionSort – Erklärung & Beispiele

InsertionSort funktioniert wie das Einsortieren von Karten:

Man baut eine sortierte Teilliste von links auf und fügt neue Elemente an der richtigen Stelle ein.

****Komplexität:**** $O(n^2)$, aber effizienter als BubbleSort für fast sortierte Daten.

****Prüfungstipp:**** Häufiger als BubbleSort in Prüfungen!

Eingabe	Schritt 1	Schritt 2	Ergebnis
[5,3,1]	[3,5,1]	[1,3,5]	[1,3,5]

SelectionSort

Pseudocode (IHK-Stil):

```
funktion selectionSort(A: Array<Integer>)  
    für i = 0 bis länge(A)-2  
        minIndex = i  
        für j = i+1 bis länge(A)-1  
            wenn A[j] < A[minIndex] dann  
                minIndex = j  
        tausche A[i], A[minIndex]  
end funktion
```

Java-Beispiel:

```
public class SelectionSortDemo {  
    static void selectionSort(int[] A) {  
        for (int i = 0; i < A.length-1; i++) {  
            int minIndex = i;  
            for (int j = i+1; j < A.length; j++) {  
                if (A[j] < A[minIndex]) minIndex = j;  
            }  
            int temp = A[i]; A[i] = A[minIndex]; A[minIndex] = temp;  
        }  
    }  
}
```

SelectionSort – Erklärung & Beispiele

SelectionSort sucht pro Durchlauf das kleinste Element und tauscht es an die richtige Stelle.

****Komplexität:**** $O(n^2)$.

****Prüfungstipp:**** Wird manchmal als Alternative zu InsertionSort abgefragt.

****Typische Fehlerquellen:****

- minIndex nicht aktualisieren
- Tauschen vergessen

Eingabe	Schritt 1	Schritt 2	Ergebnis
[5,3,1]	[1,3,5]	-	[1,3,5]

MergeSort (rekursiv)

Pseudocode (IHK-Stil):

```
funktion mergeSort(A: Array<Integer>)  
    wenn länge(A) ≤ 1 dann return A  
    mitte = länge(A)/2  
    links = mergeSort(A[0..mitte-1])  
    rechts = mergeSort(A[mitte..ende])  
    return merge(links, rechts)  
end funktion
```

Java-Beispiel:

```
public class MergeSortDemo {  
    static void mergeSort(int[] A, int l, int r) {  
        if (l < r) {  
            int m = (l+r)/2;  
            mergeSort(A, l, m);  
            mergeSort(A, m+1, r);  
            merge(A, l, m, r);  
        }  
    }  
    static void merge(int[] A, int l, int m, int r){  
        int n1 = m-l+1, n2 = r-m;  
        int[] L = new int[n1], R = new int[n2];  
        for(int i=0; i<n1; i++) L[i]=A[l+i];  
        for(int j=0; j<n2; j++) R[j]=A[m+1+j];  
        int i=0, j=0, k=l;  
        while(i<n1&& j<n2){  
            if(L[i]<=R[j]) A[k++]=L[i++];  
            else A[k++]=R[j++];  
        }  
        while(i<n1) A[k++]=L[i++];  
        while(j<n2) A[k++]=R[j++];  
    }  
}
```

MergeSort – Erklärung & Beispiele

Teile-und-herrsche-Prinzip: Array wird in zwei Hälften geteilt, rekursiv sortiert und zusammengeführt.

Komplexität: $O(n \log n)$.

Prüfungstipp: Wird eher als Theorie/Erklärung abgefragt, weniger Implementierung.

Typische Fehlerquellen:

- Merge-Funktion falsch implementiert
- Abbruchbedingung vergessen

Eingabe	Merge-Schritt 1	Merge-Schritt 2	Ergebnis
[5,3,1]	[3,5] + [1]	[1,3,5]	[1,3,5]

QuickSort (rekursiv)

Pseudocode (IHK-Stil):

```
funktion quickSort(A: Array<Integer>, links, rechts)
    wenn links < rechts dann
        p = partition(A, links, rechts)
        quickSort(A, links, p-1)
        quickSort(A, p+1, rechts)
end funktion
```

Java-Beispiel:

```
public class QuickSortDemo {
    static void quickSort(int[] A, int low, int high) {
        if (low < high) {
            int pi = partition(A, low, high);
            quickSort(A, low, pi-1);
            quickSort(A, pi+1, high);
        }
    }
    static int partition(int[] A, int low, int high) {
        int pivot = A[high];
        int i = (low-1);
        for (int j=low; j<high; j++) {
            if (A[j]<=pivot) {
                i++;
                int temp=A[i]; A[i]=A[j]; A[j]=temp;
            }
        }
        int temp=A[i+1]; A[i+1]=A[high]; A[high]=temp;
        return i+1;
    }
}
```

QuickSort – Erklärung & Beispiele

QuickSort wählt ein Pivot-Element, teilt das Array in kleinere/größere Werte auf und sortiert rekursiv.

****Komplexität:**** $\Theta O(n \log n)$, Worst Case $O(n^2)$.

****Prüfungstipp:**** Manchmal bei Theoriefragen über rekursive Sortierverfahren.

****Typische Fehlerquellen:****

- Partition falsch implementiert
- Indizes vertauscht

Eingabe	Pivot	Aufteilung	Ergebnis
[5,3,1]	1	[] + [3,5]	[1,3,5]